Lab Report Task 1: Deriving the Private Key

1. วิธีการคิดแก้ปัญหา

ใน Task นี้เป็นการคำนวณหาค่า Private Key หรือ d ของอัลกอริทิม RSA โดยอาศัยข้อมูลเบื้องต้นที่ทางโจทย์ได้กำหนดให้ ได้แก่ จำนวนเฉพาะ p, q และ Public Exponent e การแก้ปัญหาจะอ้างอิงตามทฤษฎีทางคณิตศาสตร์ของการสร้างคีย์ RSA ดังนี้

1. การคำนวณหาค่า Euler's Totient Functions (phi n)

ค่า $\emptyset(n)$ คือจำนวณของจำนวณเต็มบวกที่น้อยกว่า n และเป็นจำนวณเฉพาะสัมพัทธ์ Coprime กับ n เนื่องจากเราทราบค่า p และ q ซึ่งเป็นจำนวณเฉพาะทั้งคู่ เราจึงสามารถที่คำนวณ $\emptyset(n)$ จากสมการ

$$\emptyset(n) = (p - 1)(q - 1)$$

2. คำนวณหา Private Key

ค่า d คือ Modular Multiplicative Inverse ของ e ภายใต้โมดูโล $\emptyset(n)$ ซึ่งหมายความว่าผลคูณของ e และ d เมื่อนำไปหารด้วย $\emptyset(n)$ จะต้องเหลือเศษเป็น 1 ตามสมการ

$$e \cdot d \equiv 1 \ (mod \emptyset(n))$$

ในการอิมพลีเมนต์โปรแกรม เราจะใช้ฟังก์ชัน BN_mod_inverse() ของไลบรารี OpenSSL เพื่อคำนวณหาสมการนี้โดยตรง

2. อธิบายขั้นตอนการทำงานของโปรแกรม

โปรแกรมนี้ถูกเขียนขึ้นโดยใช้ไลบรารี `BIGNUM` ของ OpenSSL เนื่องจากตัวแปรในกระบวนการเข้ารหัส RSA มีขนาดใหญ่เกินกว่าที่ Data Type มาตรฐานของภาษา C จะรองรับได้ ในโจทย์นี้กำหนดไว้ระดับ 128-bit โดยขั้นตอนการทำงานหลักมีดังนี้

1. Initialization (`BN_CTX_new` และ `BN_new`):

เริ่มต้นด้วยการสร้างตัวแปร Context ชื่อ `BN_CTX` ซึ่งจำเป็นสำหรับจัดการหน่วยความจำชั่วคราวขณะคำนวณทางคณิตศาสตร์ จากนั้นจองพื้นที่หน่วยความจำสำหรับตัวแปร BIGNUM ต่างๆ รวมถึงการสร้างตัวแปร `one` เพื่อเก็บค่า 1 ไว้ใช้สำหรับการลบ

2. Value Assignment (ฟังก์ชั่น `BN_hex2bn`)

นำค่า p, q และ e ที่โจทย์กำหนดให้ในรูปแบบของเลขฐานสิบหก Hexadecimal เข้าไปเก็บในโครงสร้างตัวแปร BIGNUM ที่เราเตรียมไว้

3. $\emptyset(n)$ Calculation (ฟังก์ชั่น `BN_sub` และ `BN_mul`)

คำนวณหาค่า $p-1$ และ $q-1$ ด้วยฟังก์ชัน `BN_sub` (การลบกันจำนวณมากของ 2 ค่า) จากนั้นนำผลลัพธ์ทั้งสองมาคูณกันด้วยฟังก์ชัน `BN_mul` เพื่อให้ได้ค่า $\emptyset(n)$

4. Deriving Private Key (ฟังก์ชั่น `BN_mod_inverse`)

ใช้ฟังก์ชัน `BN_mod_inverse(d, e, phi, ctx)` เพื่อหาค่า Modular Inverse ของ e ภายใต้โมดูโล $\emptyset(n)$ หากสามารถหาค่าได้ (ค่า e และ $\emptyset(n)$ เป็นจำนวนเฉพาะสัมพัทธ์กัน) ค่า Private Key จะถูกเก็บไว้ในตัวแปร d และโปรแกรมจะแสดงผลลัพธ์ผ่านฟังก์ชัน `printBN`

5. Memory Cleanup (ฟังก์ชั่น `BN_free` และ `BN_CTX_free`)

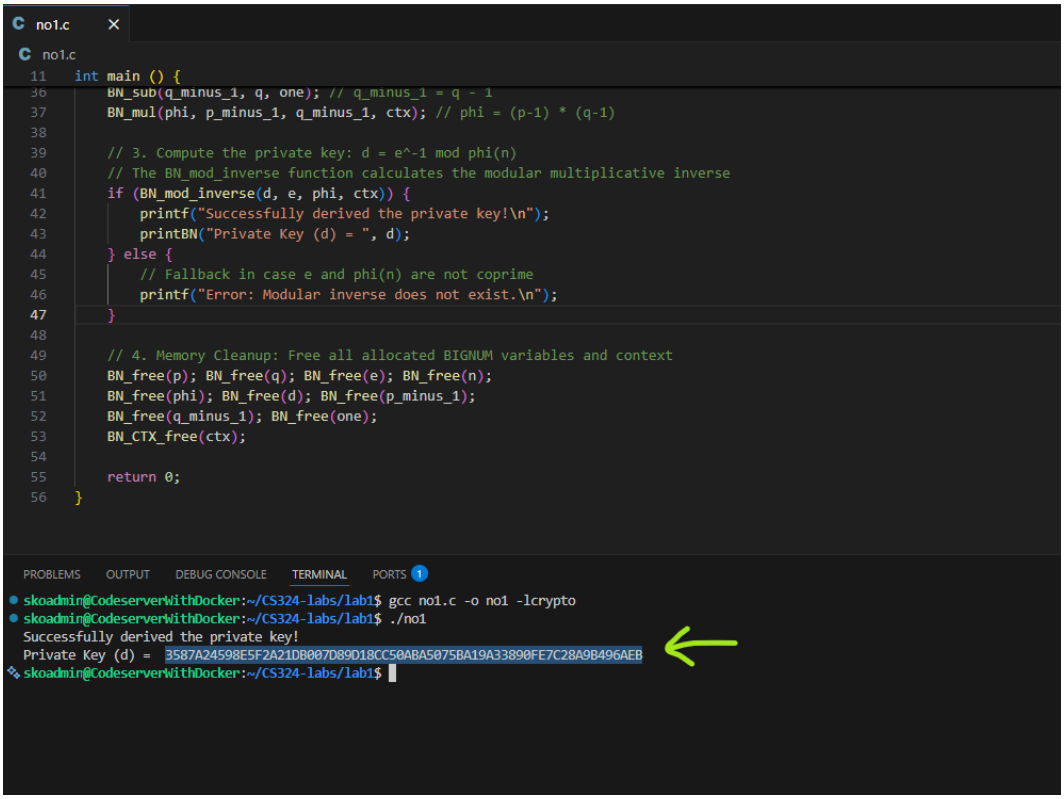
คืนพื้นที่หน่วยความจำที่ถูกจองไว้ทั้งหมดเมื่อโปรแกรมทำงานเสร็จสิ้น เพื่อป้องกันปัญหา Memory Leak

3. โค้ดการทำงาน (มีในไฟล์แยก)

```
#include <stdio.h>
#include <openssl/bn.h>

// Helper function to print a BIGNUM variable in hexadecimal format
void printBN(char *msg, BIGNUM * a) {
    char * number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str); // Free the memory allocated by BN_bn2hex
}

int main () {
    // Initialize the BIGNUM context, which is required for temporary
variables in BN operations
    BN_CTX *ctx = BN_CTX_new();

    // Allocate memory for BIGNUM variables
    BIGNUM *p = BN_new();
    BIGNUM *q = BN_new();
    BIGNUM *e = BN_new();
    BIGNUM *n = BN_new();
    BIGNUM *phi = BN_new();
    BIGNUM *d = BN_new();
```

```
    // Auxiliary variables for calculating (p-1) and (q-1)
    BIGNUM *p_minus_1 = BN_new();
    BIGNUM *q_minus_1 = BN_new();
    BIGNUM *one = BN_new();
    BN_dec2bn(&one, "1"); // Set the value of 'one' to 1 (Decimal)

    // 1. Assign given values for p, q, and e (Hexadecimal
representation)
    BN_hex2bn(&p, "F7E75FDC469067FFDC4E847C51F452DF");
    BN_hex2bn(&q, "E85CED54AF57E53E092113E62F436F4F");
    BN_hex2bn(&e, "0D88C3");

    // 2. Compute Euler's totient function: phi(n) = (p-1) * (q-1)
    BN_sub(p_minus_1, p, one); // p_minus_1 = p - 1
    BN_sub(q_minus_1, q, one); // q_minus_1 = q - 1
    BN_mul(phi, p_minus_1, q_minus_1, ctx); // phi = (p-1) * (q-1)

    // 3. Compute the private key: d = e^-1 mod phi(n)
    // The BN_mod_inverse function calculates the modular multiplicative
inverse
    if (BN_mod_inverse(d, e, phi, ctx)) {
        printf("Successfully derived the private key!\n");
        printBN("Private Key (d) = ", d);
    } else {
        // Fallback in case e and phi(n) are not coprime
        printf("Error: Modular inverse does not exist.\n");
    }

    // 4. Memory Cleanup: Free all allocated BIGNUM variables and context
    BN_free(p); BN_free(q); BN_free(e); BN_free(n);
    BN_free(phi); BN_free(d); BN_free(p_minus_1);
    BN_free(q_minus_1); BN_free(one);
    BN_CTX_free(ctx);

    return 0;
}
```

4. ผลลัพธ์การทำงาน (Execution Results)



จากการรันโปรแกรมพบว่ากระบวนการหาค่า Modular Inverse สำเร็จลุล่วง ได้ผลลัพธ์ตรงตามทฤษฎีการหาคีย์ของ RSA โดยได้ค่า Private Key d ในรูปแบบเลขฐานสิบหก ได้ค่า

```
Successfully derived the private key!
Private Key (d) =
3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB
```